

Week 6 Report

Final Touches & Release Readiness

Course: [Sum26] Practicum Project Project: BackTestBench: modular backtesting and sandbox platform for trading strategies Track: Industrial TA: Gerald Imbugwa Customer: Alexey Potemkin Date: 14.07.2026

1. Team Information

- Team name: BackTestBench
- Primary communication channel: https://t.me/Muratich_29
- Marat Diarov — Team lead, reports and customer communication, frontend.
- Georgii Beliaev — Tech lead, requirements and architecture, DevOps/QA.
- Bogdan Zinchenko — Data engineering and historical data management.
- Samy Badavi — Simulation Engine development.
- Maxim Kharlamov — Analytics and ranking.
- Philip Oghenerukevwe Idiare — Strategy Module.
- Nikolay Taran — Broker Adapter and T-Bank integration.

2. Project and Track Statement

BackTestBench is an industrial-track project for deterministic backtesting of long-only trading strategies on historical market candles. Week 5 (closed July 7) delivered MVP2: a composable YAML strategy engine, grid/random parameter optimization, Run/Stop dashboard UX, and a composable data-loader path.

Week 6 (July 8–14) expanded that surface: optimizer parameter ranking (`optimization.ranked[]`), multi-broker API tokens in the UI, composable risk controls (constraints, trailing stop, time/drawdown/trend filters), an explore dock with window-stability analytics, a minimal out-of-sample trading-bot validation pipeline, and a Binance candle adapter wired through the factory.

By July 14, 2026 the integrated product can search parameters, rank candidates, point the dashboard at multiple market-data sources, explore custom date ranges with segment stability, and spawn parallel bot validation jobs. Full multi-period walk-forward ranking for the customer's final-week ask remains planned; explore window stability is implemented as a partial slice.

3. Week 6 Goal, Scope, and Milestone Status

Week 6 is the course milestone for final touches and release readiness: finish high-priority work, run final testing, finalize documentation, and verify reproducible deployment via Docker Compose. PRs #139–#146 merged this week; documentation was synced to July 14 HEAD. Open items before customer sign-off — preset→runtime wiring, walk-forward stability column, percentage / risk-reward ranking — are scheduled for the Week 7 polish window (deadline 22 July 2026, 08:00 MSK).

Theme	Status	Evidence
Optimizer parameter ranking (ranked[])	Implemented	PR #139, src/analytics/optimizer.py
Multi-API tokens in dashboard	Implemented	PR #141, GET/POST /api/tokens
Constraints / trailing / time / guards	Implemented	PR #142
Explore dock + window stability	Implemented (partial)	PR #143/#146, src/stability.py
Minimal trading-bot validation jobs	Implemented (simulated)	PR #144, src/engine/trading_bot.py
Binance adapter + examples	Implemented	PR #145 (order/portfolio still stubs)
Frontend final UX / workflow docks	Implemented	PR #146 + July 14 fixes
Full walk-forward stability ranking	Planned	Customer P0 for Week 7 window
Preset→runtime wiring audit	Partial / open	Reported again July 13

4. Implementation Summary

4.1 Optimizer results ranking (PR #139)

- Added src/analytics/optimizer.py with OptimizerCandidate, OptimizerRankedEntry, rank_optimizer_results(), and build_optimizer_output().
- Ranks parameter combinations for one strategy (separate from strategy-level build_top_n).
- Sort order reuses analytics conventions: higher P&L, lower drawdown, higher Sharpe, higher win rate; stable tie-break.
- Serialized as optimization.ranked[] for the dashboard optimization table.
- Empty / zero-trade / non-finite metric rows are skipped.

4.2 Multi-API tokens and data-source UI (PR #141)

- Dashboard gained token management for multiple broker sources (T-Bank, TwelveData, Bybit; later Binance).
- Live routes: GET/POST /api/tokens.
- Adapter factory (src/broker_adapter/factory.py) resolves source → adapter and env var names (TOKEN_ENV_BY_SOURCE).
- Per-tab market pickers for explore/bot allow instrument and API selection without rebuilding the Python image.

4.3 Composable constraints, trailing stop, and filters (PR #142)

- Extended the composable engine with constraints, time predicates, trailing stop, drawdown guard, and trend filter.
- Supports customer-discussed exit/filter design: time-based session close, higher-timeframe trend awareness, and priority-ordered exit rules (TP/SL remain first-class in YAML).
- Covered by unit tests under tests/unit/strategy/composable/ (test_constraints, test_trailing_stop, test_time_predicates, test_guard_and_trend).

4.4 Explore dock and window stability (PR #143 / #146) — partial

- Explore workflow: `POST /api/explore → main.py explore-job → data/explore-jobs/*.json`; UI polls `GET /api/explore`.
- Custom date range and advanced analytics for strategy exploration beyond the default lookback bootstrap.
- `src/stability.py` computes window-segment stability from equity-curve chart points (return segments, comparison vs benchmark). This is not full multi-period walk-forward ranking across prior months with a stability column in Top-N — that remains the final-week P0.
- Frontend workflow dock integrates explore alongside backtest and bot controls.

4.5 Minimal trading bot validation pipeline (PR #144)

- `src/engine/trading_bot.py` runs out-of-sample validation on recent market data using the same execution engine and analytics validation helpers.
- Job path: `POST /api/bot → main.py bot-job → data/bot-jobs/*.json`; UI polls `GET /api/bot`.
- Parallel bot jobs reuse the data-loader / SQLite cache path where possible.
- Not a live broker trading scheduler: fills are simulated; T-Bank `place_order / get_portfolio` remain `NotImplementedError`.

4.6 Binance adapter, examples, and frontend final (PR #145 / #146 + fixes)

- `BinanceAdapter` added to the factory for spot klines (alongside T-Bank, TwelveData, Bybit).
- Examples documentation refreshed for multi-platform usage.
- Despite PR #145's title mentioning sandbox `place_order / get_portfolio`, those methods in `src/broker_adapter/tbank.py` still raise `NotImplementedError` as of July 14.
- Frontend final version (#146) plus follow-up fixes (July 14): expanded preset ranges, trading-bot corrections, backtest control-panel load speedup, `crypto.randomUUID` polyfill for non-secure contexts.

5. Updated API Surface

Implemented Next.js App Router routes as of July 14, 2026:

Method / path	Role
POST /api/bootstrap	Start main backtest bootstrap
GET/POST /api/config	Dashboard run configuration
GET /api/dashboard	Poll backtest runtime state
POST /api/stop	Stop running Python job
GET/POST /api/strategies, DELETE /api/strategies/[id]	Strategy catalogue CRUD
POST /api/refresh-ranking	Rebuild strategy Top-N
GET/POST /api/tokens	Multi-API token store (Week 6)
GET/POST /api/explore	Explore jobs + poll (Week 6)
GET/POST /api/bot	Trading-bot validation jobs + poll (Week 6)

Runtime bridge remains JSON files (data/runtime-dashboard.json, data/explore-jobs/, data/bot-jobs/), not a durable relational run store. Planned FastAPI service in src/api is still not the live integration path.

6. Industrial Track Contribution

Dimension	Before (Week 3 MVP, June 2026)	After (Week 6, July 2026)
Strategy model	Single hardcoded MA Crossover	Composable YAML rules engine with TP/SL, constraints, trailing, filters
Data sources	T-Bank only	Factory: T-Bank, TwelveData, Bybit, Binance; token UI in dashboard
User workflow	One-shot run, static dashboard	Run/Stop backtest, optimizer grid/random + ranked[], explore dock, trading-bot validation
Ranking	Strategy Top-N on backtest metrics only	Strategy Top-N + per-strategy optimizer parameter ranking
Validation	Analytics library only	Explore window stability + out-of-sample bot jobs (simulated fills)
Documentation	Week 4 audit baseline	Living docs synced July 14; README, API, Docker, architecture matrix
CI / release	Self-hosted Docker smoke	GitHub-hosted ubuntu-latest: pytest + frontend build + docker-smoke
Deployment	Developer-local scripts	docker compose up --build as the supported integrated runtime

7. Screenshots / Demo Evidence

All captures taken from the live dashboard on July 14, 2026 (<http://localhost:3000>, runtime data/runtime-dashboard.json).

Multi-API data sources and run settings (PR #141)

TRADING ANALYTICS

BackTestBench

Instrument: BTCUSDTTF: 15mSource: Binance

FIND STRATEGY

Go to

Enter a name or ID and press Enter

BACKTEST CONTROL

Run settings

Run backtest

Stop

DATA SOURCE

☐ T-BANKREADY

MOEX TQBR shares via Tinkoff Invest API

Saved as t.7x...l93w

☐ TWELVE DATA

TOKEN REQUIRED

Global equities, FX and crypto via Twelve Data API

Select to add API token

☐ BYBITPUBLIC API

Crypto spot pairs via Bybit public kline API

No API key needed

☒ BINANCEPUBLIC API

Crypto spot pairs via Binance public kline API

No API key needed

INSTRUMENT

BTCUSDT

Crypto spot pairs (BTCUSDT, ETHUSDT, ...)

TIMEFRAME

15m

Min resolution 1m. Max lookahead for 15m: 3650 days

LOOKBACK DAYS

24

History window loaded for the backtest

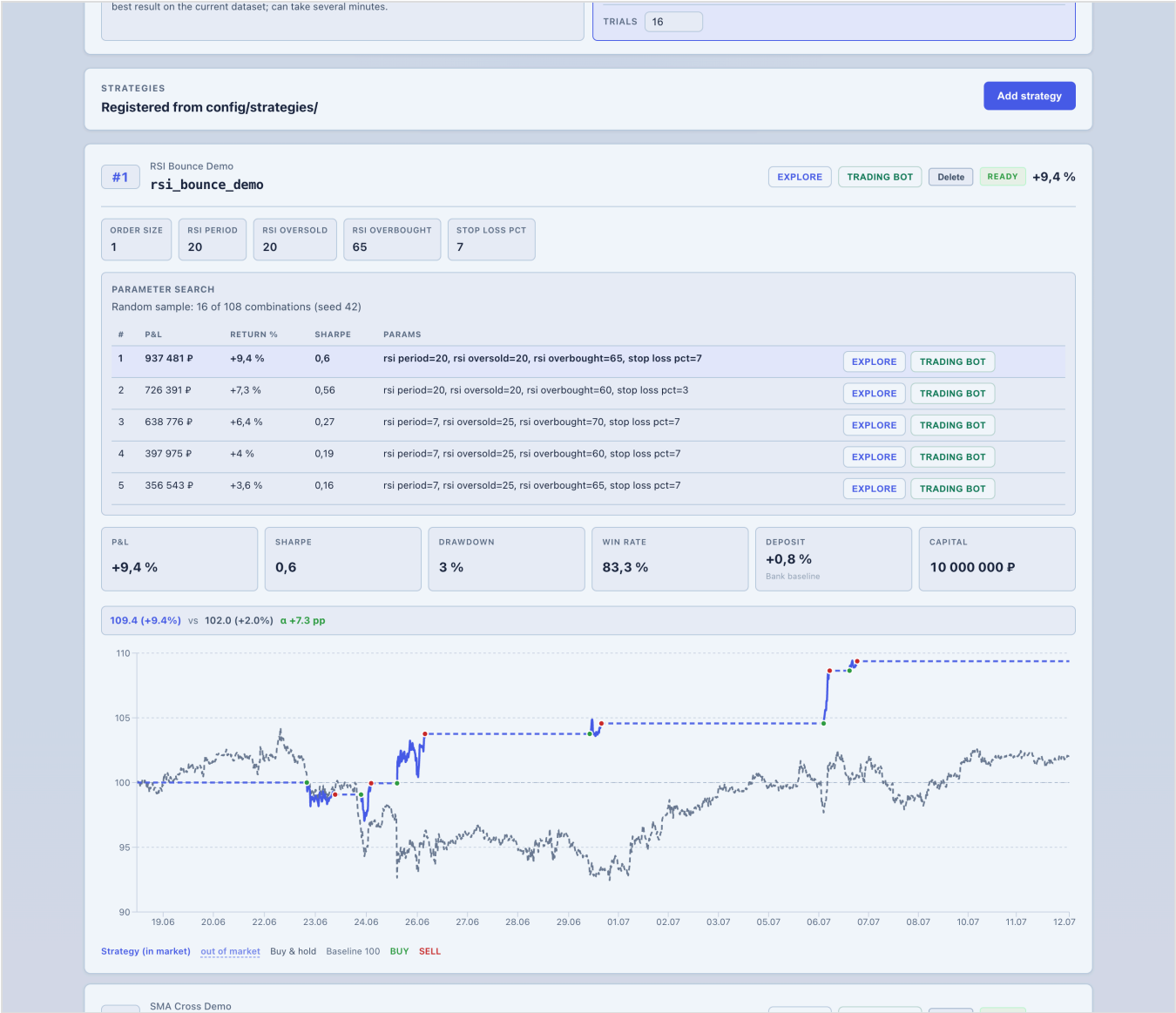
INITIAL CAPITAL (RUB)

10000000

Starting portfolio cash for each strategy

BackTestBench — data source cards and instrument/timeframe configuration

Optimizer ranking for one strategy (PR #139)



BackTestBench — RSI Bounce Demo with parameter search table, metrics, and equity chart

Add-strategy YAML editor

GRID

Full grid search - evaluates every valid parameter combination from YAML choices. Guarantees the best result on the current dataset; can take several minutes.

SAMPLE

Random sample - evaluates N random combinations (see iterations + seed). Faster, but does not guarantee the global optimum.

TRIALS 16

STRATEGIES
Registered from config/strategies/ Close editor

STRATEGY YAML

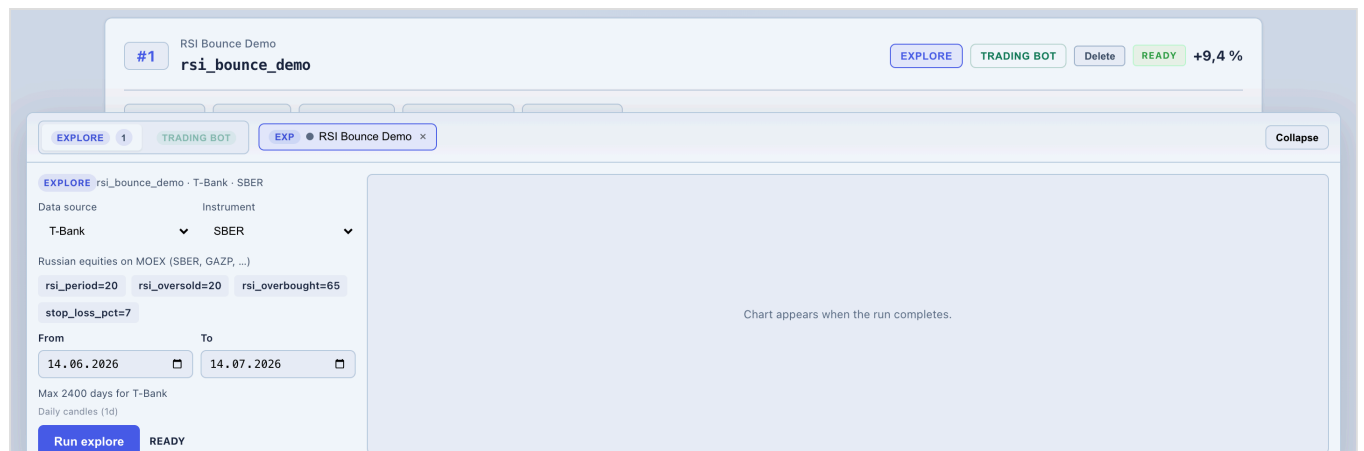
```
# Composable strategy YAML (saved to config/strategies/{name}.yaml)
name: my_strategy
title: My Strategy

params:
  order_size: { type: float, default: 1, choices: [1, 2, 3], optimizable: false }

series:
  sma_fast: { fn: sma, source: price, period: 10 }
  sma_slow: { fn: sma, source: price, period: 30 }
```

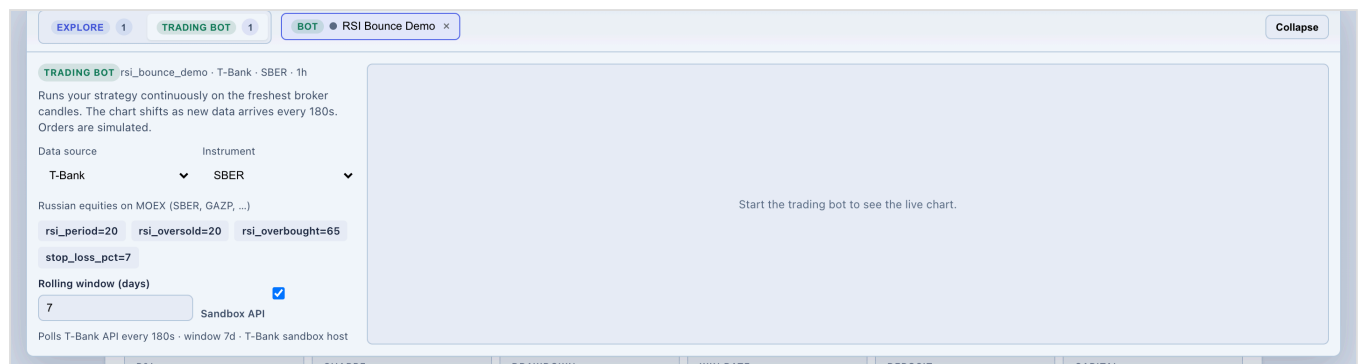
BackTestBench — composable strategy YAML editor with Save / Cancel

Explore workflow dock (PR #143 / #146)



BackTestBench — Explore dock with date range and Run explore controls

Trading bot validation dock (PR #144)



BackTestBench — Trading Bot tab with rolling out-of-sample validation session

Backend test and coverage evidence (July 14, 2026)

```

usr/registry/metric_registry.py      41  0  100%
usr/registry/metric.py               42  0  100%
usr/registry/notification_engine.py   62  1  94%
usr/registry/notification.py          29  0  100%
usr/registry/pool_executor.py         29  0  100%
usr/registry/poolfillin.py            77  0  100%
usr/registry/reading_buffer.py        44  0  100%
usr/registry/types.py                 5  0  100%
usr/registry.py                      43  0  100%
usr/notification/__init__.py          0  0  100%
usr/notification/metric.py            0  0  100%
usr/notification/__init__.py          0  0  100%
usr/notification/scheduler.py         0  0  100%
usr/stability.py                     60  0  93%
usr/strategy/__init__.py              12  0  100%
usr/strategy/basic.py                 12  0  100%
usr/strategy/composable/__init__.py   12  0  100%
usr/strategy/composable/metric.py     26  0  100%
usr/strategy/composable/complex.py    61  1  99%
usr/strategy/composable/complex.py    37  0  100%
usr/strategy/composable/context.py    41  11  82%
usr/strategy/composable/default.py     66  7  89%
usr/strategy/composable/error.py       114  0  100%
usr/strategy/composable/predict.py     114  10  91%
usr/strategy/composable/reset.py       26  0  100%
usr/strategy/composable/rule.py        11  0  100%
usr/strategy/composable/step.py        113  0  95%
usr/strategy/composable/strategy.py    52  0  96%
usr/strategy/complex.py               46  0  100%
usr/strategy/complex.py               5  0  100%
usr/strategy/discard.py               27  0  100%
usr/strategy/registry.py              36  4  89%
usr/strategy/scheme.py                56  0  100%
usr/strategy/scheme.py               45  0  87%
usr/strategy/strategy/__init__.py     3  0  100%
usr/strategy/strategy/metric_resolver.py 147  7  94%
usr/strategy/strategy/metric.py       48  11  84%
usr/strategy/strategy/metric_threshold.py 69  11  84%
usr/strategy/strategy.py              49  2  96%
usr/strategy_validation.py            43  11  23%

TOTAL                                188  18  71%
=====

```

pytest summary — 188 passed, 77% src coverage, July 14, 2026

CI workflow (GitHub-hosted ubuntu-latest)

```
name: PR Build & Run

on:
  pull_request:
    branches: [main]

jobs:
  backend-tests:
    runs-on: ubuntu-latest
    if: github.event.action != 'closed'

    steps:
      - name: Checkout PR
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: requirements.txt

      - name: Install Python dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install -r requirements.txt

      - name: Run backend tests
        run: pytest tests -q

  frontend-checks:
    runs-on: ubuntu-latest
    if: github.event.action != 'closed'

    steps:
      - name: Checkout PR
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm
          cache-dependency-path: frontend/package-lock.json

      - name: Install and build frontend
        working-directory: frontend
        run: |
          npm ci
          npm run build

      - name: Lint frontend (non-blocking until page.tsx refactor)
        working-directory: frontend
        continue-on-error: true
        run: npm run lint

  docker-smoke:
    runs-on: ubuntu-latest
    if: github.event.action != 'closed'

    env:
      APP_PORT: "8080"

    steps:
      - name: Checkout PR
        uses: actions/checkout@v4

      - name: Prepare .env for Compose
        run: |
          cat > .env <<EOF
          TINKOFF_TOKEN=${{ secrets.TINKOFF_TOKEN }}
          TWELVEDATA_TOKEN=${{ secrets.TWELVEDATA_TOKEN }}
          EOF

      - name: Build and start via docker compose
        run: docker compose up -d --build

      - name: Smoke-check (containers up + HTTP 200)
        run: |
          sleep 10
          docker compose ps
          EXITED=$(docker compose ps --status exited -q)
          [ -z "$EXITED" ]
          curl -fsS "http://localhost:${APP_PORT}/"

      - name: Tear down stack
        if: always()
        run: docker compose down --remove-orphans 2>/dev/null || true
```

GitHub Actions ci.yml — backend-tests, frontend-checks, docker-smoke on ubuntu-latest

8. Customer Review and Feedback

Review date

Composable engine / optimizer demo followed up with the customer on ~13.07.2026 (transcriptions/13-07-26-customer.txt). Direction echoes the July 6 session (transcriptions/06-07-26-customer.txt) with sharper final-week priorities.

Positive confirmation

- Rules-based strategies work end-to-end (triggers, actions, sizing, TP/SL from rules, priority exits).
- File-driven definitions accepted: new strategies parse without rewriting the engine.
- Grid and random optimization understood; random seed+batch accepted as a way to sample large combinatorial spaces.
- Parameter model (defaults vs optimizable ranges, nested series) reviewed positively.

Issues confirmed in demo

- Preset wiring: changing param_presets.yaml (or UI values) does not always change the executed backtest; some strategies appear display-only for presets.
- Percent semantics / leverage confusion around stop-loss percentages.
- Optimizer UX still too engineering-facing (seed/batch) for non-technical users; table updates after full batch, not incrementally.
- Stop/run and polling felt chatty during live demo.

Customer direction for the final week

P0 — Must have

1. Fix preset → runtime wiring so TP/SL and strategy params actually apply; one source of truth; surface active TP/SL.
2. Multi-period stability check for top-N optimized configs (at least 2–3 prior months / rolling windows).
3. Stability column (or score) in the optimization / ranking table.
4. Percentage-based metrics alongside absolute P&L.

P1 — Should have

1. Persistent sidebar of active strategy parameters.
2. Chart: dashed/gap when flat vs in position.
3. Simpler optimizer UX ("N random trials" instead of exposing seed/batch).
4. Risk/reward ratio in metrics; optional sort/filter.

P2 — Stretch / post-course

1. "Investigate" drill-down (trades, YoY).
2. Shorts + higher-TF trend filters; market-regime switching.
3. Live trading bot mirroring backtest rules; futures support.

Customer priority for the final week: move from "best params on one month" to proving those params are stable over time.

9. Updated Backlog

Completed in Week 6

- Optimizer parameter ranking schema and ranked[] (#139).
- Multi-API token insertion and dashboard wiring (#141).
- Composable constraints, time filters, trailing stop, drawdown/trend guards (#142).
- Explore workflow + window stability analytics (#143/#146; partial vs full walk-forward).
- Minimal trading-bot out-of-sample validation jobs (#144).
- Binance candle adapter + examples (#145 content; order APIs still stubs).
- Frontend final version and runtime/UI fixes (#146, July 14 commits).
- Living documentation audited to July 14 HEAD.

Next priority (final course week)

- Preset→runtime audit (P0): which strategies read presets at execution vs UI-only; fix mismatches.
- Walk-forward / multi-period stability (P0): run top configs on prior slices; add stability column to ranking — beyond explore window segments.
- Percentage and risk/reward metrics (P0–P1): return %, drawdown %, capital used %, R:R; ranking not by P&L alone.
- UX: persistent active params, flat-vs-in-market chart styling, simpler random-optimizer controls.

Deferred / known gaps

- Full end-to-end holdout validation as a dedicated second stage (library + bot jobs exist; not a separate formal stage).
- T-Bank place_order / get_portfolio (still stubs).
- Relational run-history persistence; FastAPI service; multi-instrument portfolio UI; scheduler / notifications / live order automation.
- Short selling and futures.

10. Required Links

Repository and planning:

- Repository: <https://github.com/BackTest-bench-team/BackTestBench>
- Kanban board: <https://github.com/orgs/BackTest-bench-team/projects/1>
- Issues: <https://github.com/BackTest-bench-team/BackTestBench/issues>

Pull requests merged this week (July 8–14):

- #139 — Optimizer results schema and parameter ranking (ranked[])
- #141 — Multi-API / token insertion in the frontend
- #142 — Composable constraints, time filters, trailing stop, drawdown/trend guards
- #143 — Frontend final version (explore / advanced analytics slice)
- #144 — Minimal trading bot validation pipeline
- #145 — Binance adapter and examples (T-Bank order/portfolio remain stubs)
- #146 — Frontend final version (workflow docks and UX completion)

Notable follow-up commits on July 14: trading-bot fixes, control-panel load speedup, `crypto.randomUUID` polyfill, expanded preset ranges.

Documentation:

- Status matrix: `docs/README.md` (audited July 14, 2026)
- Customer notes: `transcriptions/13-07-26-customer.txt` (also `06-07-26-customer.txt` for Week 5→6 goals)
- Living README: `root README.md`

11. Verification

Commands run on July 14, 2026:

- Backend: 188 tests passed — `pytest tests -q`
- Coverage: 77% of `src/` — `pytest tests --cov=src --cov-report=term`
- Frontend: production build succeeded — `npm --prefix frontend run build`
- CI: GitHub-hosted `ubuntu-latest` jobs `backend-tests`, `frontend-checks`, `docker-smoke` (PR #137)

Non-blocking warnings:

- Frontend build prints Turbopack path-join warnings from `spawn-python.ts` / `next.config.ts` (build still succeeds).
- Trading bot module coverage is still low (~30% line coverage) relative to core engine modules.

Conclusion

Week 6 delivered the Final Touches & Release Readiness milestone: composable strategies, optimizer ranking, multi-API data, explore and bot validation, finalized docs, Docker one-command deployment, and CI green on `ubuntu-latest`. The July 13 customer review confirmed the direction toward multi-period stability; preset wiring and walk-forward ranking remain for the Week 7 polish window.